

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO



FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y DISEÑO DIGITAL CARRERA DE INGENIERÍA EN SOFTWARE

TEMA:

GA – EXPOSICIÓN SEGUNDO CORTE: HERRAMIENTAS CASE (UMBRELLO UML MODELLER)
Ciclo MDD: modelo UML, generación de código C#, verificación y roundtrip

DOCUMENTO INDIVIDUAL:

INVESTIGACIÓN INDIVIDUAL - FRIXON MORÁN

ASIGNATURA:

INGENIERÍA DE REQUISITOS (ISR-401)

DOCENTE:

ING. GLEISTON GUERRERO ULLOA, PhD.

ESTUDIANTE:

MORÁN PILAGUANO FRIXON FERNANDO

RESPONSABILIDAD PRINCIPAL:

Modelado UML, normalización del subsistema, atributos, operaciones, relaciones, cardinalidades y trazabilidad.

EQUIPO:

A

CURSO/PARALELO:

CUARTO NIVEL PARALELO “B”

SISTEMA PFC:

SISTEMA INTELIGENTE DE CONTROL Y SEGUIMIENTO DE TERAPIA FÍSICA

HERRAMIENTA MDD Y LENGUAJE OBJETIVO:

UMBRELLO UML MODELLER – C#

REPOSITORIO GITHUB:

<https://github.com/AngeloP2114/SISTEMA-DE-TERAPIA-EQUIPOA-UMBRELLO-UML/tree/main>

QUEVEDO – LOS RÍOS – ECUADOR

2026 – 2027 PPA

Índice

1. Resumen	1
2. Introducción	1
3. Delimitación del subsistema	1
4. Requisitos funcionales que originan el modelo	1
5. Modelo construido en Umbrello	2
6. Diseño de las clases	3
6.1. Usuario	3
6.2. Paciente	3
6.3. Fisioterapeuta.....	4
6.4. PlanTerapia	4
6.5. Ejercicio.....	4
6.6. SesionTerapia.....	4
6.7. EvaluacionProgreso	4
7. Catálogo consolidado de atributos	4
8. Catálogo consolidado de operaciones	5
9. Relaciones UML y cardinalidades	5
9.1. Generalización	5
9.2. Asociaciones	5
9.3. Agregación.....	6
9.4. Composición.....	6
10. Revisión de consistencia	6
10.1. Correcciones detectadas	6
11. Mapeo esperado hacia C#	6
12. Aporte personal de Frixon	7
13. Conclusiones	7
Apéndice A: Procedimiento de construcción en Umbrello	7
Apéndice B: Lista de verificación del modelo	7
Apéndice C: Mejoras futuras del modelo	8
Apéndice D: Guion personal para Frixon	8
Apéndice E: Preguntas que Frixon debe dominar	8

1. Resumen

Esta investigación documenta el aporte de Frixon Morán en la construcción y revisión del modelo UML de origen utilizado para generar código C# con Umbrello UML Modeller. El trabajo se aplica al Sistema Inteligente de Control y Seguimiento de Terapia Física y comprende la delimitación del subsistema, la identificación de clases del dominio, la definición de atributos tipados, operaciones con firmas completas, visibilidad, relaciones y cardinalidades. Se analizan las siete clases del modelo: *Usuario*, *Paciente*, *Fisioterapeuta*, *PlanTerapia*, *Ejercicio*, *SesionTerapia* y *EvaluacionProgreso*. También se estudian los criterios para utilizar generalización, asociación, agregación y composición, así como el mapeo esperado hacia C#. La investigación compara la captura real de Umbrello con una versión normalizada y explica las correcciones necesarias, entre ellas la eliminación de nombres temporales como *new_attribute*, la unificación de tipos y la revisión de multiplicidades. El aporte de Frixon constituye la base técnica del ciclo MDD, porque la calidad del código generado depende directamente de la calidad sintáctica y semántica del modelo.

2. Introducción

El diagrama de clases es el artefacto obligatorio para la generación estructural de código. Su propósito no es solo mostrar cajas conectadas, sino representar conceptos del dominio mediante elementos UML con semántica definida. Una clase debe contener responsabilidades, atributos, operaciones y relaciones coherentes; de lo contrario, el generador trasladará al código los errores o ambigüedades existentes en el modelo [1, 2].

El subsistema modelado pertenece al PFC de terapias físicas. El objetivo es representar el flujo básico desde la identificación de usuarios hasta la planificación, ejecución y evaluación de una terapia. La selección de siete clases supera el mínimo exigido por la rúbrica y permite mostrar varios mecanismos de UML: herencia, asociaciones, agregación, composición y multiplicidades [3].

La responsabilidad de Frixon se concentra en organizar el dominio, construir las clases, revisar la notación y preparar un modelo procesable por Umbrello. Este aporte es determinante: la generación de Angelo y la fundamentación de Esther dependen de que el modelo de origen sea consistente.

3. Delimitación del subsistema

El sistema completo de terapias físicas podría incluir citas, expedientes, autenticación, notificaciones, pagos, reportes, almacenamiento multimedia y análisis de movimiento. Sin embargo, un modelo para una demostración MDD debe ser representativo y controlable. El alcance seleccionado cubre:

- identificación de usuarios del sistema;
- especialización de pacientes y fisioterapeutas;
- creación de planes de terapia;
- inclusión de ejercicios;
- programación y registro de sesiones;
- evaluación del progreso del paciente.

Esta delimitación evita un modelo trivial, pero también impide que la exposición se vuelva inmanejable. El subsistema contiene suficiente información para evidenciar correspondencia con requisitos y generar siete archivos C#.

4. Requisitos funcionales que originan el modelo

Tabla 1: Requisitos funcionales considerados durante el modelado

ID	Descripción	Elemento principal
RF-01	Registrar información general de los usuarios.	Usuario.
RF-02	Mantener información clínica básica del paciente.	Paciente.
RF-03	Registrar especialidad y licencia del fisioterapeuta.	Fisioterapeuta.
RF-04	Crear planes con fechas, objetivo y estado.	PlanTerapia.
RF-05	Asociar ejercicios con repeticiones y duración.	Ejercicio.
RF-06	Registrar sesiones con fecha, duración y observaciones.	SesionTerapia.
RF-07	Evaluar dolor, movilidad, fatiga y progreso.	EvaluacionProgreso.

La identificación de requisitos antes de crear las clases evita introducir elementos por relleno. La trazabilidad exige que cada clase tenga una razón de existir y que sus miembros aporten a una necesidad del PFC [4, 5].

5. Modelo construido en Umbrello

La Figura 1 muestra la captura real del modelo desarrollado. Esta evidencia es importante porque demuestra que las clases y relaciones se encuentran dentro de Umbrello y no fueron dibujadas en una herramienta externa.

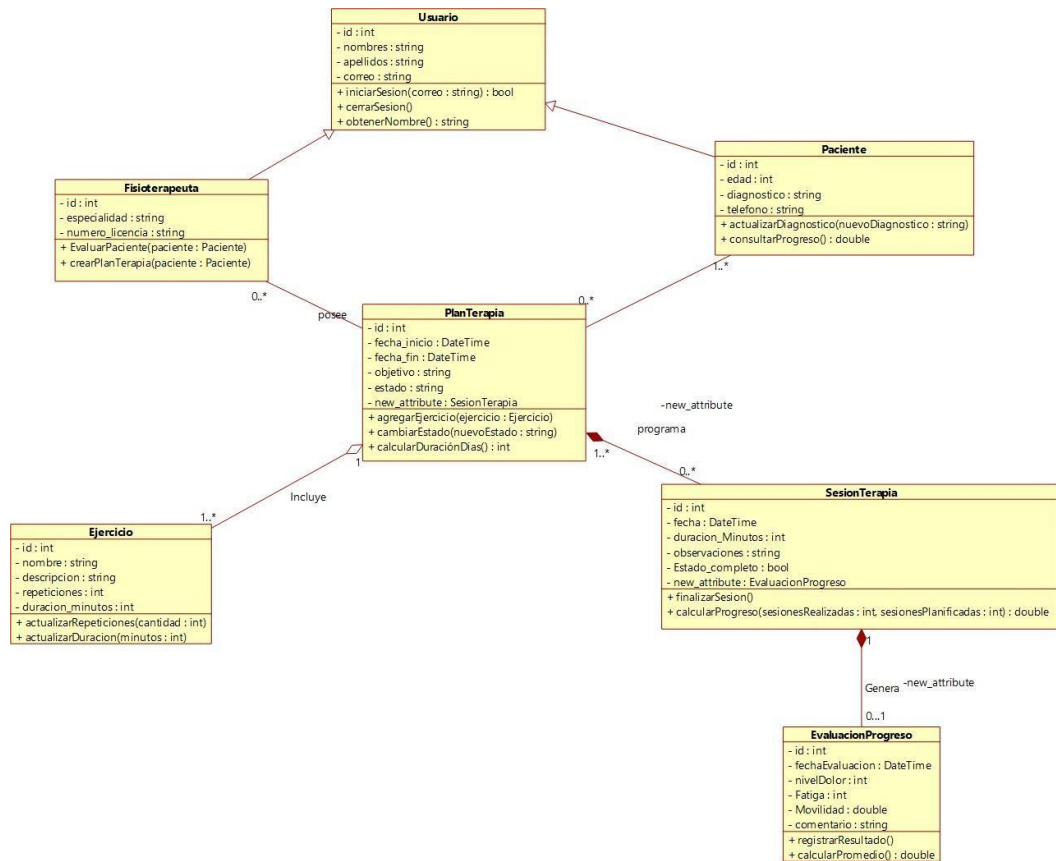


Figura 1: Modelo original construido en Umbrello UML Modeller.

La captura registra siete clases, herencia, asociaciones, agregación y composición. Durante la revisión se identificaron elementos temporales `new_attribute` asociados a algunas relaciones. Estos nombres no representan requisitos del sistema y deben eliminarse o sustituirse por nombres semánticos antes de considerar el modelo final.

La Figura 2 muestra una representación normalizada utilizada para revisar la estructura sin textos temporales ni cruces innecesarios.

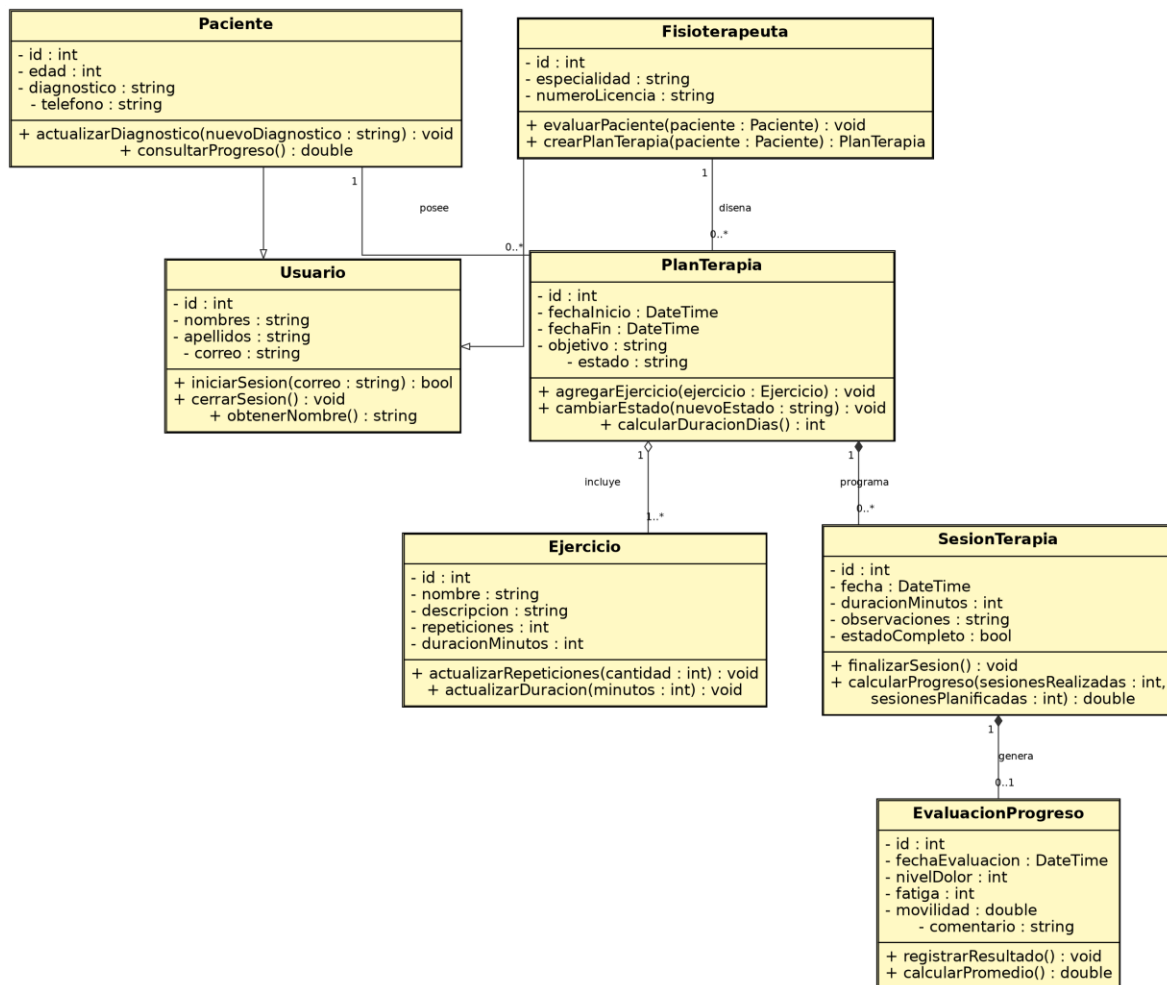


Figura 2: Representación normalizada del modelo de terapia física.

6. Diseño de las clases

6.1. Usuario

Usuario contiene datos compartidos por los actores que acceden al sistema. Centralizar nombres, apellidos y correo evita duplicación en las clases especializadas.

Tabla 2: Especificación de la clase Usuario

Miembro	Tipo/retorno	Justificación
idUsuario	int	Identificador interno.
nombres, apellidos	string	Datos de identificación.
correo	string	Medio de acceso o contacto.
iniciarSesion	bool	Indica si las credenciales son válidas.
cerrarSesion	void	Finaliza la interacción autenticada.
obtenerNombreCompleto	string	Devuelve una representación legible.

La clase funciona como generalización de **Paciente** y **Fisioterapeuta**. En UML, el triángulo vacío apunta hacia la clase más general [1].

6.2. Paciente

Paciente representa a la persona que recibe tratamiento. Los datos heredados no deben repetirse. Sus atributos propios son edad, diagnóstico y teléfono. La operación `actualizarDiagnostico(nuevoDiagnostico:string):void` modifica información clínica, mientras que `consultarProgreso():double` representa una consulta cuantitativa.

6.3. Fisioterapeuta

La clase almacena especialidad y número de licencia. Sus operaciones reciben objetos del dominio:

- `evaluarPaciente(paciente:Paciente):void;`
- `crearPlanTerapia(paciente:Paciente):PlanTerapia.`

La expresión `paciente:Paciente` no es un error. El término en minúscula es el nombre del parámetro y el término con mayúscula es su tipo. Recibir un objeto completo permite utilizar los datos y comportamiento de la clase.

6.4. PlanTerapia

`PlanTerapia` organiza el tratamiento. Contiene identificador, fechas, objetivo y estado. Las fechas utilizan `DateTime`, creado como datatype en Umbrello cuando no aparece entre los tipos predeterminados. Entre sus operaciones se encuentran agregar ejercicios, cambiar estado y calcular duración.

6.5. Ejercicio

La clase describe una actividad terapéutica mediante nombre, descripción, repeticiones y duración en minutos. La presencia de `duracionMinutos` también en `SesionTerapia` es válida porque los atributos pertenecen a clases diferentes. En `Ejercicio` representa la duración de una actividad; en `SesionTerapia`, la duración total de la sesión.

6.6. SesionTerapia

La clase registra fecha, duración, observaciones y estado de finalización. La operación `calcularProgreso(sesionesRealizadas:int, sesionesPlanificadas:int):double` modela una firma que después requiere implementación en C#.

6.7. EvaluacionProgreso

Esta clase registra indicadores como nivel de dolor, movilidad, fatiga y comentario. Se separa de la sesión para representar una entidad con información clínica propia. La multiplicidad `0..1` indica que una sesión puede no haber sido evaluada todavía o puede producir una evaluación.

7. Catálogo consolidado de atributos

Todos los atributos fueron definidos con visibilidad privada. Esta decisión protege el estado interno y permite que las modificaciones se realicen mediante operaciones controladas.

Tabla 3: Atributos del modelo de origen

Clase	Atributo	Tipo	Visibilidad
Usuario	idUsuario	int	Private
Usuario	nombres	string	Private
Usuario	apellidos	string	Private
Usuario	correo	string	Private
Paciente	edad	int	Private
Paciente	diagnostico	string	Private
Paciente	telefono	string	Private
Fisioterapeuta	especialidad	string	Private
Fisioterapeuta	numeroLicencia	string	Private
PlanTerapia	idPlan	int	Private
PlanTerapia	fechaInicio	DateTime	Private
PlanTerapia	fechaFin	DateTime	Private
PlanTerapia	objetivo	string	Private
PlanTerapia	estado	string	Private
Ejercicio	idEjercicio	int	Private
Ejercicio	nombre	string	Private

Clase	Atributo	Tipo	Visibilidad
Ejercicio	descripcion	string	Private
Ejercicio	repeticiones	int	Private
Ejercicio	duracionMinutos	int	Private
SesionTerapia	idSesion	int	Private
SesionTerapia	fecha	DateTime	Private
SesionTerapia	duracionMinutos	int	Private
SesionTerapia	observaciones	string	Private
SesionTerapia	completada	bool	Private
EvaluacionProgreso	idEvaluacion	int	Private
EvaluacionProgreso	fechaEvaluacion	DateTime	Private
EvaluacionProgreso	nivelDolor	int	Private
EvaluacionProgreso	fatiga	int	Private
EvaluacionProgreso	movilidad	double	Private
EvaluacionProgreso	comentario	string	Private

8. Catálogo consolidado de operaciones

Las operaciones se definieron con visibilidad pública para representar servicios accesibles desde otros objetos. Los parámetros se agregaron mediante la sección de parámetros de Umbrello y no como parte del nombre del método.

Tabla 4: Operaciones principales del modelo

Clase	Operación	Retorno	Visibilidad
Usuario	iniciarSesion(correo:string)	bool	Public
Usuario	cerrarSesion()	void	Public
Usuario	obtenerNombre()	string	Public
Paciente	actualizarDiagnostico(nuevoDiagnostico:svtroidg)		Public
Paciente	consultarProgreso()	double	Public
Fisioterapeuta	evaluarPaciente(paciente:Paciente)	void	Public
Fisioterapeuta	crearPlanTerapia(paciente:Paciente)	PlanTerapia	Public
PlanTerapia	agregarEjercicio(ejercicio:Ejercicio)	void	Public
PlanTerapia	cambiarEstado(nuevoEstado:string)	void	Public
PlanTerapia	calcularDuracionDias()	int	Public
Ejercicio	actualizarRepeticiones(cantidad:int)	void	Public
Ejercicio	actualizarDuracion(minutos:int)	void	Public
SesionTerapia	finalizarSesion()	void	Public
SesionTerapia	calcularProgreso(realizadas:int, planificadas:int)	double	Public
EvaluacionProgreso	registrarResultado()	void	Public
EvaluacionProgreso	calcularPromedio()	double	Public

9. Relaciones UML y cardinalidades

9.1. Generalización

Paciente y *Fisioterapeuta* heredan de *Usuario*. Esta relación evita duplicar atributos comunes. Para crearla en Umbrello se selecciona *Generalization*, se hace clic primero en la clase hija y luego en la clase padre.

9.2. Asociaciones

Las asociaciones representan vínculos sin dependencia fuerte de ciclo de vida:

- un paciente posee cero o varios planes;
- un fisioterapeuta diseña cero o varios planes.

En ambos casos, cada plan se vincula con un paciente y un fisioterapeuta. La multiplicidad $0..*$ permite que un actor exista antes de tener planes asociados.

9.3. Agregación

La relación entre `PlanTerapia` y `Ejercicio` se modela con rombo blanco en el lado del plan. El ejercicio puede existir como elemento de catálogo y reutilizarse. La multiplicidad `1..*` expresa que un plan debe incluir al menos un ejercicio.

9.4. Composición

El plan compone sesiones y la sesión compone una evaluación. El rombo negro se coloca junto al todo. La composición indica una relación más fuerte: la sesión se gestiona dentro del plan, y la evaluación pertenece al contexto de la sesión.

Tabla 5: Relaciones y multiplicidades definitivas

Origen	Destino	Tipo	Multiplicidades
Paciente	Usuario	Generalización	No aplica.
Fisioterapeuta	Usuario	Generalización	No aplica.
Paciente	PlanTerapia	Asociación	Paciente 1; Plan 0..*.
Fisioterapeuta	PlanTerapia	Asociación	Fisio 1; Plan 0..*.
PlanTerapia	Ejercicio	Agregación	Plan 1; Ejercicio 1..*.
PlanTerapia	SesionTerapia	Composición	Plan 1; Sesión 0..*.
SesionTerapia	EvaluacionProgreso	Composición	Sesión 1; Evaluación 0..1.

10. Revisión de consistencia

La validación del modelo incluyó una revisión manual porque algunas versiones de Umbrello no muestran un único comando de validación semántica para todos los elementos. Se aplicó la siguiente lista:

- las siete entidades fueron creadas como clases reales;
- los tipos auxiliares se registraron como datatypes;
- cada operación tiene retorno explícito;
- los atributos no se repiten dentro de una misma clase;
- las generalizaciones apuntan a `Usuario`;
- los rombos se encuentran en el lado del todo;
- las asociaciones incluyen multiplicidades;
- los roles temporales se eliminaron;
- se evitaron nombres con espacios o tildes en elementos destinados al código.

10.1. Correcciones detectadas

La captura original presenta elementos `new_attribute` vinculados a `PlanTerapia` y `SesionTerapia`. Estos campos pueden surgir cuando una asociación genera atributos automáticos o cuando se confirma una creación sin renombrar. Deben corregirse porque reducen la legibilidad y pueden producir campos genéricos en C#.

También se revisaron nombres como `Estado_completo` y `duracion_Minutos`. Para mantener una convención uniforme se recomienda `completada` y `duracionMinutos`. La normalización de nombres es parte de la calidad del modelo, no un detalle estético.

11. Mapeo esperado hacia C#

Tabla 6: Decisiones de mapeo UML–C#

Elemento UML	Construcción C# esperada
Clase	Archivo y declaración <code>public class</code> .
Atributo privado	Campo <code>private</code> .
Operación pública	Método <code>public</code> .
Generalización	<code>class Paciente : Usuario</code> .
Parámetro de clase	<code>Ejercicio ejercicio o Paciente paciente</code> .
Multiplicidad múltiple	Colección tipada, si el generador la materializa.
Composición/agregación	Referencia o colección con semántica documentada.

El mapeo de relaciones puede variar según la configuración y versión de Umbrello. Por ello, la tabla distingue el resultado esperado de la comprobación real. La trazabilidad debe documentar lo que efectivamente aparece en los archivos, no solo lo que se esperaba.

12. Aporte personal de Frixon

El aporte individual comprende:

1. delimitación del subsistema representativo;
2. identificación de siete clases del dominio;
3. creación de atributos y operaciones;
4. definición de tipos primitivos y `DateTime`;
5. configuración de visibilidad privada y pública;
6. creación de generalizaciones, asociaciones, agregación y composición;
7. asignación de multiplicidades;
8. organización visual del diagrama;
9. revisión de elementos temporales y nombres inconsistentes;
10. elaboración de tablas de clases, relaciones y trazabilidad;
11. explicación del modelo durante la exposición.

Esta contribución demuestra que el modelo no fue copiado de un tutorial genérico, sino construido para el PFC y preparado para funcionar como origen de generación.

13. Conclusiones

La calidad del modelo UML condiciona directamente la calidad de la salida. Un tipo incorrecto, una operación sin retorno o una relación mal orientada puede producir código incompleto o confuso. El modelo del sistema de terapias físicas contiene siete clases relevantes y representa un flujo coherente desde el usuario hasta la evaluación del progreso.

La normalización permitió identificar y corregir nombres temporales, convenciones inconsistentes y posibles ambigüedades. Las relaciones seleccionadas expresan diferentes grados de dependencia: la herencia concentra atributos comunes, las asociaciones vinculan actores y planes, la agregación permite reutilizar ejercicios y la composición representa partes controladas por el plan o la sesión.

El aporte de Frixon constituye la base del proceso MDD. Sin un modelo correcto no es posible demostrar una generación fiable, establecer trazabilidad ni realizar un roundtrip controlado.

Apéndice A

Procedimiento de construcción en Umbrello

La siguiente secuencia resume el procedimiento reproducible que Frixon debe conocer y explicar:

1. crear el diagrama dentro de *Logical View*;
2. crear las siete clases como elementos *Class*;
3. registrar `DateTime` dentro de *Datatypes*;
4. añadir cada atributo indicando nombre, tipo y visibilidad *Private*;
5. añadir cada operación indicando retorno y visibilidad *Public*;
6. agregar parámetros mediante *New Parameter*;
7. crear generalización haciendo clic primero en la clase hija y luego en *Usuario*;
8. crear asociaciones y configurar nombre y multiplicidad desde *Properties*;
9. ubicar el rombo blanco de agregación junto a *PlanTerapia*;
10. ubicar los rombos negros de composición junto al elemento que representa el todo;
11. dejar vacíos los nombres de roles si no se necesitan para la generación;
12. agrandar las cajas y añadir puntos de control para evitar cruces;
13. guardar el archivo nativo y exportar una imagen legible.

Apéndice B

Lista de verificación del modelo

Tabla 7: Checklist de calidad del modelo de origen

Comprobación	Estado esperado	Evidencia
Existen al menos seis clases del PFC.	Cumple	Siete clases.

Comprobación	Estado esperado	Evidencia
Cada atributo posee tipo.	Cumple	Propiedades de atributos.
Cada operación posee retorno.	Cumple	Firmas UML.
Los parámetros tienen nombre y tipo.	Cumple	Operaciones del modelo.
Los atributos son privados.	Cumple	Símbolo “-”.
Las operaciones son públicas.	Cumple	Símbolo “+”.
Las generalizaciones apuntan a Usuario.	Cumple	Triángulo vacío.
Las asociaciones tienen multiplicidad.	Cumple	1, 0..*, 1..*, 0..1.
El rombo de agregación está en el todo.	Cumple	PlanTerapia–Ejercicio.
Los rombos de composición están en el todo.	Cumple	Plan–Sesión y Sesión–Evaluación.
No existen nombres temporales.	Debe corregirse	Eliminar new_attribute.
Los nombres siguen una convención uniforme.	Debe revisarse	camelCase sin guiones bajos.
El archivo nativo está en GitHub.	Obligatorio	Carpeta modelo/.

Apéndice C

Mejoras futuras del modelo

El modelo puede evolucionar sin afectar el alcance de la demostración actual. Entre las mejoras posibles se encuentran:

- incorporar enumeraciones para el estado del plan y de la sesión;
- representar una clase `DetallePlanEjercicio` si se necesita configurar repeticiones específicas por plan;
- separar los datos de autenticación de la información personal;
- agregar restricciones sobre nivel de dolor, fechas y porcentajes;
- modelar una máquina de estados para `PlanTerapia`;
- añadir un diagrama de secuencia para registrar una sesión;
- incorporar estereotipos de persistencia solo si el generador los utiliza.

Estas propuestas no deben agregarse por relleno. Cada elemento futuro debe corresponder a un requisito y aportar a la comprensión o generación.

Apéndice D

Guion personal para Frixon

La exposición del modelo puede organizarse de la siguiente manera:

1. presentar el alcance y justificar las siete clases;
2. mostrar que `Paciente` y `Fisioterapeuta` heredan de `Usuario`;
3. explicar por qué los atributos son privados y los métodos públicos;
4. aclarar parámetros como `paciente:Paciente` y `ejercicio:Ejercicio`;
5. explicar asociación, agregación y composición usando ejemplos del dominio;
6. interpretar las multiplicidades;
7. señalar las correcciones realizadas para normalizar el modelo;
8. entregar el hilo a Angelo: “Con el modelo validado, ahora se generan las clases C#.”

Apéndice E

Preguntas que Frixon debe dominar

1. ¿Por qué `duracionMinutos` puede repetirse? Porque pertenece a clases diferentes y representa conceptos distintos.
2. ¿Por qué un ejercicio es agregación y una sesión composición? El ejercicio puede reutilizarse; la sesión se gestiona como parte del plan específico.
3. ¿Qué significa `0..1`? Puede no existir una evaluación todavía o existir una sola.

4. **¿Por qué no se repiten nombres y correo en Paciente?** Se heredan de Usuario.
5. **¿Qué ocurre si una operación no tiene retorno?** El código generado puede ser ambiguo o no compilar según la firma.

Referencias

- [1] Object Management Group, “OMG Unified Modeling Language (OMG UML), Version 2.5.1,” Object Management Group, inf. téc. formal/17-12-05, 2017. dirección: <https://www.omg.org/spec/UML/2.5.1>.
- [2] M. Brambilla, J. Cabot y M. Wimmer, *Model-Driven Software Engineering in Practice*, 2.^a ed. Morgan & Claypool Publishers, 2017. DOI: 10.2200/S00751ED2V01Y201701SWE001.
- [3] G. G. Ulloa, *Guía y Rúbrica de Evaluación: Exposición-Demostración MDD (UML a Código)*, Universidad Técnica Estatal de Quevedo, 2026.
- [4] ISO/IEC/IEEE, “ISO/IEC/IEEE 29148:2018: Systems and Software Engineering – Life Cycle Processes – Requirements Engineering,” inf. téc., 2018. dirección: <https://www.iso.org/standard/72089.html>.
- [5] O. C. Z. Gotel y A. C. W. Finkelstein, “An Analysis of the Requirements Traceability Problem,” en *Proceedings of the First International Conference on Requirements Engineering*, 1994, págs. 94-101. DOI: 10.1109/ICRE.1994.292398.